

## #Part 1 – Dataset Understanding

```
In [7]: # Load the dataset with pandas and display the first 5 rows.  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
df = pd.read_csv("Mall_Customers.csv")  
  
df.head()
```

Out[7]:

|   | CustomerID | Gender | Age | Annual<br>Income (k\$) | Spending Score<br>(1-100) |
|---|------------|--------|-----|------------------------|---------------------------|
| 0 | 1          | Female | 29  | 24                     | 75                        |
| 1 | 2          | Male   | 37  | 87                     | 37                        |
| 2 | 3          | Male   | 42  | 57                     | 44                        |
| 3 | 4          | Female | 34  | 16                     | 68                        |
| 4 | 5          | Male   | 22  | 16                     | 74                        |

```
In [8]: # Print the dataset shape, column data types, and check for missing
values.
print("Shape:", df.shape)

print("\nData Types")
print(df.dtypes)

print("\nMissing Values")
print(df.isnull().sum())
```

Shape: (220, 5)

Data Types

|                        |        |
|------------------------|--------|
| CustomerID             | int64  |
| Gender                 | object |
| Age                    | int64  |
| Annual Income (k\$)    | int64  |
| Spending Score (1-100) | int64  |
| dtype:                 | object |

Missing Values

|                        |       |
|------------------------|-------|
| CustomerID             | 0     |
| Gender                 | 0     |
| Age                    | 0     |
| Annual Income (k\$)    | 0     |
| Spending Score (1-100) | 0     |
| dtype:                 | int64 |

```
In [9]: #Show the distribution of Gender and basic statistics (describe()) for
        numeric columns.

        print(df["Gender"].value_counts())
        df.describe()
```

```
Gender
Female    123
Male      97
Name: count, dtype: int64
```

Out[9]:

|       | CustomerID | Age        | Annual Income<br>(k\$) | Spending Score<br>(1-100) |
|-------|------------|------------|------------------------|---------------------------|
| count | 220.000000 | 220.000000 | 220.000000             | 220.000000                |
| mean  | 110.500000 | 38.568182  | 54.468182              | 48.177273                 |
| std   | 63.652704  | 10.394057  | 26.674837              | 27.224090                 |
| min   | 1.000000   | 18.000000  | 15.000000              | 1.000000                  |
| 25%   | 55.750000  | 30.000000  | 27.000000              | 22.000000                 |
| 50%   | 110.500000 | 38.000000  | 53.000000              | 50.000000                 |
| 75%   | 165.250000 | 47.000000  | 80.250000              | 74.000000                 |
| max   | 220.000000 | 62.000000  | 102.000000             | 100.000000                |

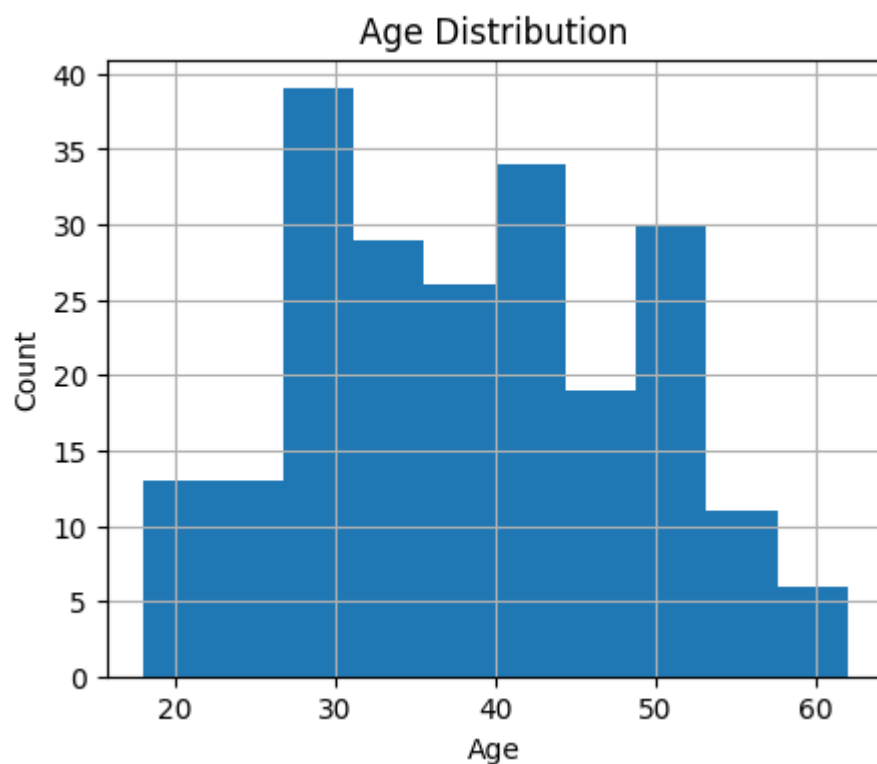
##Write 2–3 short observations about what the data looks like.

Observations-

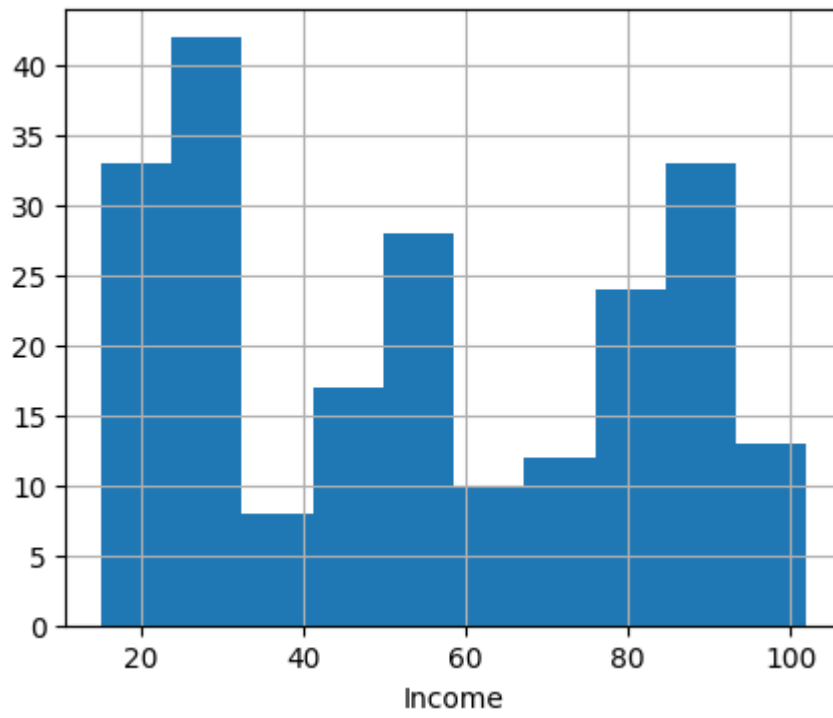
1. The dataset contains 220 customer records and 5 columns. It is a clean dataset with no missing values, so no preprocessing is required before analysis.
2. The dataset has both categorical and numerical features. Gender is the only categorical column, while Age, Annual Income, and Spending Score are numerical. CustomerID is just used to identify each customer.
3. Looking at the data, customers have different income levels and spending habits. This suggests that they can be grouped into different customer segments using clustering techniques.

#Part 2 – Exploratory Data Analysis (EDA)

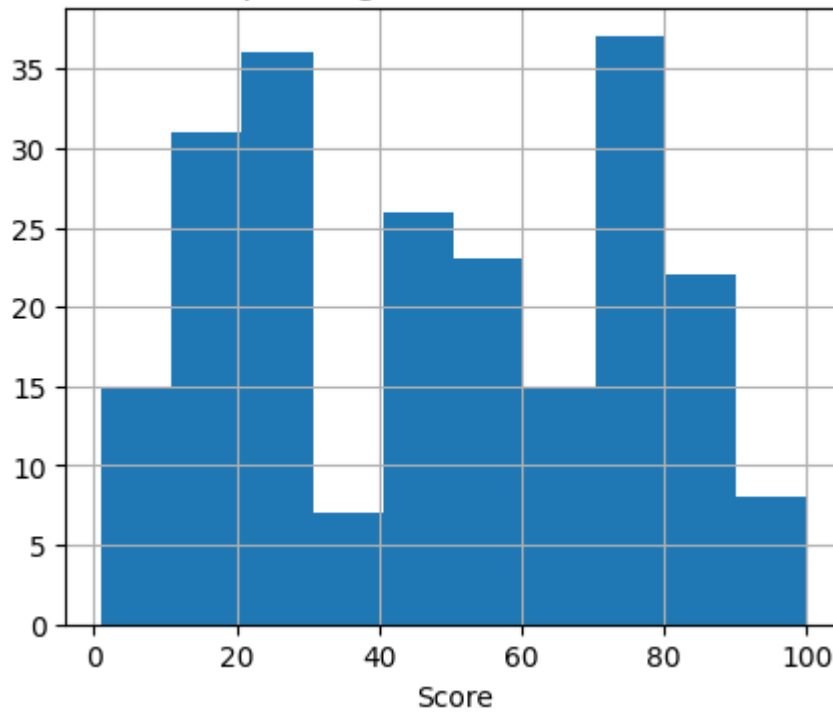
```
In [10]: #Plot histograms for Age, Annual Income, and Spending Score.  
df["Age"].hist(figsize=(5,4))  
plt.title("Age Distribution")  
plt.xlabel("Age")  
plt.ylabel("Count")  
plt.show()  
  
df["Annual Income (k$)"].hist(figsize=(5,4))  
plt.title("Annual Income Distribution")  
plt.xlabel("Income")  
plt.show()  
  
df["Spending Score (1-100)"].hist(figsize=(5,4))  
plt.title("Spending Score Distribution")  
plt.xlabel("Score")  
plt.show()
```



Annual Income Distribution



Spending Score Distribution



```
In [11]: #Draw a scatter plot of Annual Income (x-axis) vs Spending Score (y-
axis).
plt.figure(figsize=(6,5))
plt.scatter(df["Annual Income (k$)"],
            df["Spending Score (1-100)"])

plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("Income vs Spending Score")
plt.show()
```



**Looking at the scatter plot, roughly how many natural groups can you see? Write your guess.**

Based on the scatter plot of Annual Income vs Spending Score, I can roughly identify 5 natural groups of customers. These groups appear to represent customers with different income levels and spending behaviors, such as high-income/high-spending, high-income/low-spending, low-income/high-spending, low-income/low-spending, and average-income/average-spending customers.

```
In [15]: #Standardise (scale) the two clustering features using StandardScaler
from sklearn.preprocessing import StandardScaler

X = df[["Annual Income (k$)", "Spending Score (1-100)"]]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print(X_scaled[:5])
```

```
[[ -1.14481167  0.98750396]
 [  1.22235076 -0.41150182]
 [  0.09513055 -0.15379023]
 [-1.44540372  0.72979237]
 [-1.44540372  0.95068801]]
```

###explain in one line why scaling matters for distance-based methods.

Distance-based algorithms such as K-Means, KNN, and DBSCAN calculate the distance between data points to make decisions. If one feature has much larger values than another the larger-valued feature will have a greater influence on the distance calculation. Scaling brings all features to a similar range, ensuring that each feature contributes equally and improves the accuracy of the results.

#Part 3 – K-Means Clustering

```
In [16]: #Run K-Means on the scaled Income & Spending features.
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=5, random_state=42)
kmeans.fit(X_scaled)

df["Cluster"] = kmeans.labels_
df.head()
```

Out[16]:

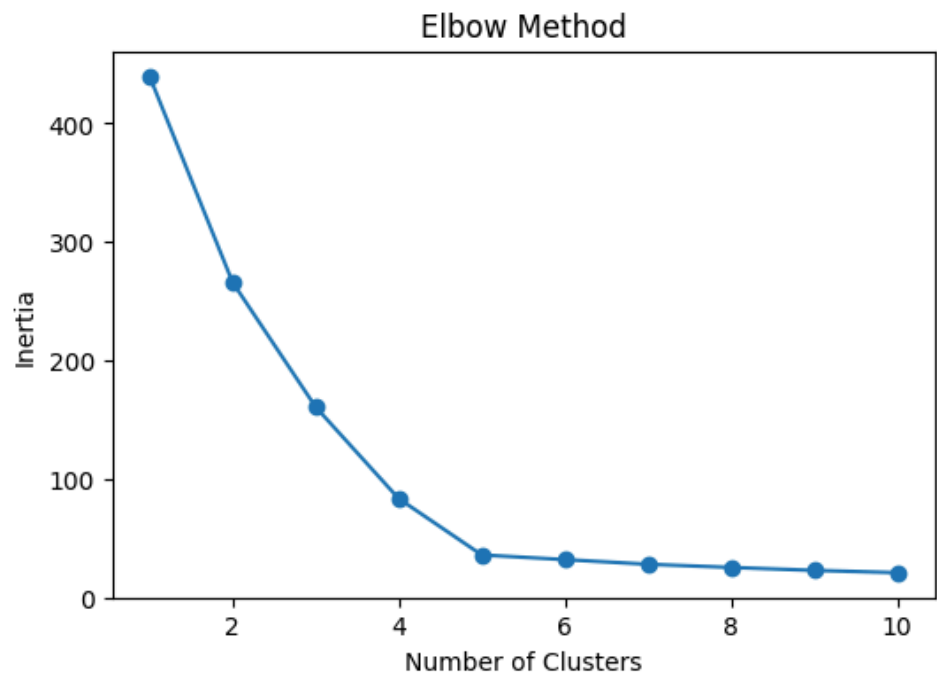
|   | CustomerID | Gender | Age | Annual<br>Income<br>(k\$) | Spending<br>Score (1-<br>100) | Cluster |
|---|------------|--------|-----|---------------------------|-------------------------------|---------|
| 0 | 1          | Female | 29  | 24                        | 75                            | 3       |
| 1 | 2          | Male   | 37  | 87                        | 37                            | 4       |
| 2 | 3          | Male   | 42  | 57                        | 44                            | 1       |
| 3 | 4          | Female | 34  | 16                        | 68                            | 3       |
| 4 | 5          | Male   | 22  | 16                        | 74                            | 3       |

```
In [18]: # Use the Elbow Method (plot inertia for k = 1 to 10) to pick a good
number of clusters k.
from sklearn.cluster import KMeans

inertia = []

for k in range(1,11):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    inertia.append(model.inertia_)

plt.figure(figsize=(6,4))
plt.plot(range(1,11), inertia, marker='o')
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.show()
```





```
In [19]: #compute the Silhouette Score for k = 2 to 7 and confirm your choice of k.
from sklearn.metrics import silhouette_score

for k in range(2,8):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = model.fit_predict(X_scaled)

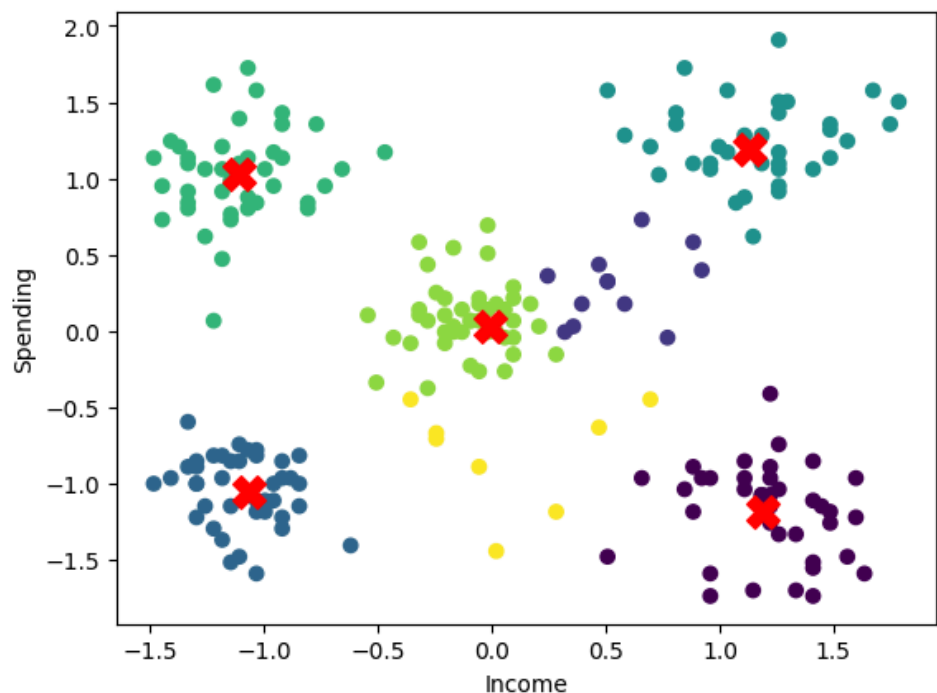
    score = silhouette_score(X_scaled, labels)

    print("k =", k, "Silhouette Score =", round(score,3))

k = 2 Silhouette Score = 0.388
k = 3 Silhouette Score = 0.478
k = 4 Silhouette Score = 0.563
k = 5 Silhouette Score = 0.655
k = 6 Silhouette Score = 0.577
k = 7 Silhouette Score = 0.567
```

```
In [24]: #Fit the final K-Means model and make a scatter plot with each cluster
in a different colour, marking the
plt.scatter(X_scaled[:,0], X_scaled[:,1], c=labels)
plt.scatter(kmeans.cluster_centers[:,0],
            kmeans.cluster_centers[:,1],
            c='red', marker='X', s=200)

plt.xlabel("Income")
plt.ylabel("Spending")
plt.show()
```



####Briefly describe each customer segment you found (e.g. 'high income, low spending').

#### Customer Segments-

Cluster 0: Customers with average income and average spending, representing regular shoppers.

Cluster 1: Customers with high income and high spending, who are premium and loyal customers.

Cluster 2: Customers with low income and high spending, who spend actively despite having lower incomes.

Cluster 3: Customers with high income and low spending, who have good purchasing power but spend less and can be targeted through marketing campaigns.

Cluster 4: Customers with low income and low spending, who are budget-conscious and contribute less to overall sales.

#### #Part 4 – DBSCAN Clustering

```
In [32]: # Run DBSCAN on the same scaled features.
from sklearn.cluster import DBSCAN
dbscan = DBSCAN(eps=0.5, min_samples=5)
db_labels = dbscan.fit_predict(X_scaled)
df["DBSCAN Cluster"] = db_labels

print(df.head())
```

|   | CustomerID | Gender | Age | Annual Income (k\$) | Spending Score (1-100) |
|---|------------|--------|-----|---------------------|------------------------|
| 0 | 1          | Female | 29  | 24                  |                        |
| 1 | 2          | Male   | 37  | 87                  |                        |
| 2 | 3          | Male   | 42  | 57                  |                        |
| 3 | 4          | Female | 34  | 16                  |                        |
| 4 | 5          | Male   | 22  | 16                  |                        |

|   | Cluster | KMeans Cluster | DBSCAN Cluster |
|---|---------|----------------|----------------|
| 0 | 3       | 3              | 0              |
| 1 | 4       | 4              | 1              |
| 2 | 1       | 1              | 2              |
| 3 | 3       | 3              | 0              |
| 4 | 3       | 3              | 0              |

```
In [25]: #Try a few values of eps (e.g. 0.3, 0.4, 0.5) with min_samples = 5.
from sklearn.cluster import DBSCAN

for eps in [0.3,0.4,0.5]:

    db = DBSCAN(eps=eps,
                min_samples=5)

    labels = db.fit_predict(X_scaled)

    print("\nEPS =", eps)
    print("Clusters:",
          len(set(labels))-(1 if -1 in labels else 0))
    print("Noise:",
          list(labels).count(-1))
```

```
EPS = 0.3
Clusters: 5
Noise: 13
```

```
EPS = 0.4
Clusters: 5
Noise: 6
```

```
EPS = 0.5
Clusters: 4
Noise: 0
```

```
In [26]: #Plot the DBSCAN clusters, showing noise points (label = -1) in a
          separate colour.
          db = DBSCAN(eps=0.5,
                      min_samples=5)

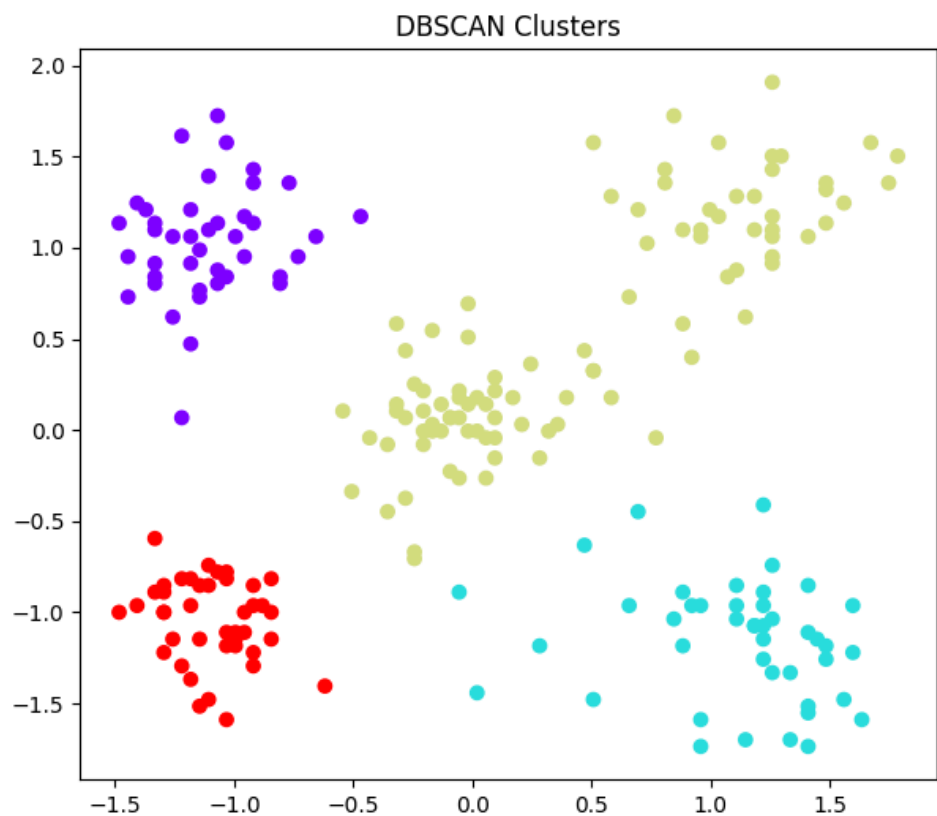
          db_labels = db.fit_predict(X_scaled)

          df["DBSCAN Cluster"] = db_labels

          plt.figure(figsize=(7,6))

          plt.scatter(X_scaled[:,0],
                      X_scaled[:,1],
                      c=db_labels,
                      cmap="rainbow")

          plt.title("DBSCAN Clusters")
          plt.show()
```



```
In [33]: #State how many clusters and how many noise/outlier points DBSCAN found.
n_clusters = len(set(db_labels)) - (1 if -1 in db_labels else 0)

n_noise = list(db_labels).count(-1)

print("Number of Clusters Found:", n_clusters)
print("Number of Noise/Outlier Points:", n_noise)

print("\nCluster Labels:")
print(df["DBSCAN Cluster"].value_counts().sort_index())
```

```
Number of Clusters Found: 4
Number of Noise/Outlier Points: 0
```

```
Cluster Labels:
DBSCAN Cluster
0      41
1      43
2      95
3      41
Name: count, dtype: int64
```

##State how many clusters and how many noise/outlier points DBSCAN found.  
 From the DBSCAN clustering results, we found that the algorithm identified 4 customer clusters and no noise (outlier) points. This means that all 220 customers were successfully assigned to one of the four clusters, and no data points were considered isolated or located in low-density regions. Hence, DBSCAN was able to cluster the entire dataset without detecting any outliers.

#Part 5 – Comparing K-Means vs DBSCAN

```
In [27]: #Put the two cluster plots side by side
fig, ax = plt.subplots(1,2,figsize=(12,5))

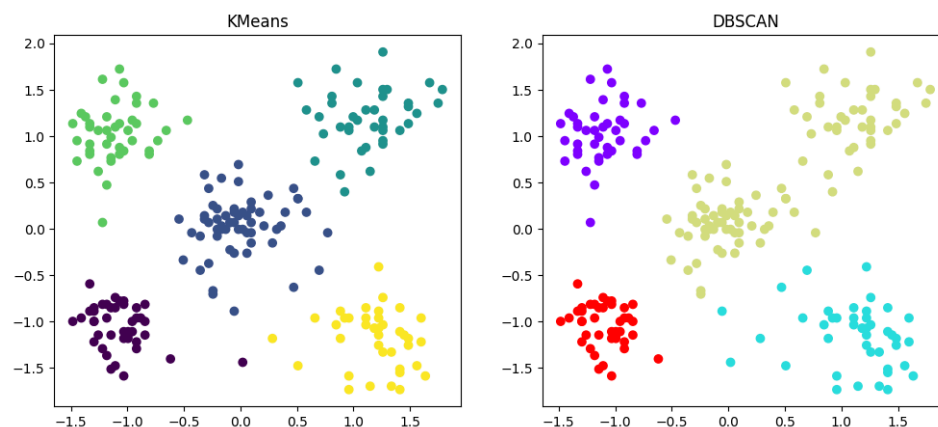
ax[0].scatter(X_scaled[:,0],
              X_scaled[:,1],
              c=clusters,
              cmap="viridis")

ax[0].set_title("KMeans")

ax[1].scatter(X_scaled[:,0],
              X_scaled[:,1],
              c=db_labels,
              cmap="rainbow")

ax[1].set_title("DBSCAN")

plt.show()
```



###Fill a small comparison table: number of clusters found, whether outliers are detected, and whether you must choose k in advance.

**Feature | K-Means | DBSCAN |**

| ----- | ----- | ----- |

**Number of Clusters Found** | 5 | 4 |

**Detects Outliers** | No | No (0 outliers found in this dataset) |

**Need to Choose k in Advance** | Yes | No |

###Write 3–4 sentences on when you would prefer one method over the other.

K-Means is preferred when the approximate number of clusters is already known and the data forms clear, well-separated groups. It is simple, fast, and works efficiently on large datasets. DBSCAN is preferred when the number of clusters is unknown or when the dataset may contain noise and outliers, as it can identify them automatically. In this dataset, K-Means produced clearer customer segments, while DBSCAN grouped the data into fewer clusters without detecting any outliers.

#Part 6 – Overfitting and Underfitting

```
In [35]: #Use your K-Means cluster labels as the 'segment' each customer belongs
to.
X = X_scaled

y = df["KMeans Cluster"]

print("Features shape:", X.shape)
print("Target shape:", y.shape)

print("\nFirst 10 customer segments:")
print(y.head(10))
```

```
Features shape: (220, 2)
Target shape: (220,)

First 10 customer segments:
0    3
1    4
2    1
3    3
4    3
5    1
6    4
7    1
8    3
9    4
Name: KMeans Cluster, dtype: int32
```

```
In [36]: # Train a KNN classifier to predict the segment from the features, using
a train/test split.
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

X = X_scaled

y = df["KMeans Cluster"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

print("KNN classifier trained successfully.")
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
KNN classifier trained successfully.
Training samples: 176
Testing samples: 44
```

```
In [29]: #Vary the number of neighbours k (e.g. 1, 3, 5, 15, 30, 80) and record
train accuracy and test accuracy for each.
neighbors = [1,3,5,15,30,80]

train_acc = []
test_acc = []

for k in neighbors:

    knn = KNeighborsClassifier(n_neighbors=k)

    knn.fit(X_train,y_train)

    train_pred = knn.predict(X_train)
    test_pred = knn.predict(X_test)

    train_acc.append(
        accuracy_score(y_train,train_pred)
    )

    test_acc.append(
        accuracy_score(y_test,test_pred)
    )

print(train_acc)
print(test_acc)
```

```
[1.0, 0.9943181818181818, 1.0, 0.9943181818181818, 1.0,
0.2897727272727273]
[1.0, 1.0, 1.0, 1.0, 1.0, 0.25]
```



```
In [30]: # Plot train vs test accuracy against k. Mark which region is
overfitting (very small k) and which is underfitting (very large k).
plt.figure(figsize=(7,5))

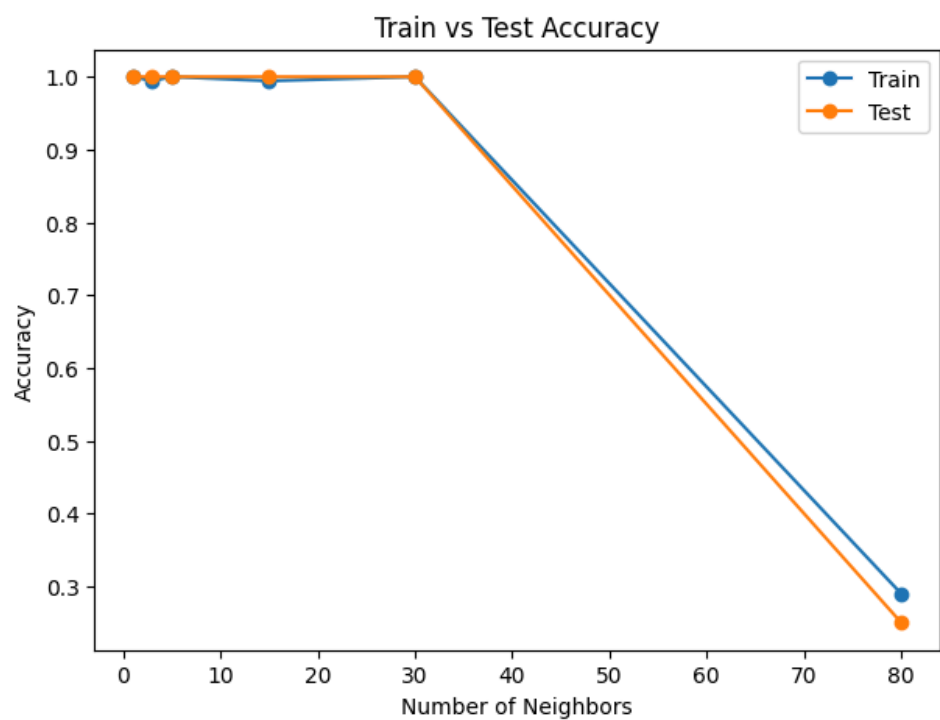
plt.plot(neighbors,
         train_acc,
         marker='o',
         label="Train")

plt.plot(neighbors,
         test_acc,
         marker='o',
         label="Test")

plt.xlabel("Number of Neighbors")
plt.ylabel("Accuracy")
plt.title("Train vs Test Accuracy")

plt.legend()

plt.show()
```



###In 2–3 sentences, explain the pattern you see.

The graph shows that both training and testing accuracy remain very high for smaller values of  $k$  (1–30), indicating good model performance. However, when  $k$  is increased to 80, both accuracies drop sharply, suggesting underfitting because the model becomes too simple and fails to classify the customer segments accurately.

#Part 7 – Comparative Analysis & Recommendation

###Summarise your findings in one final table (method, key setting, result).

| Method             | Key Setting                | Result                                                                                               |
|--------------------|----------------------------|------------------------------------------------------------------------------------------------------|
| K-Means Clustering | k = 5                      | Identified <b>5 customer segments</b> based on income and spending score.                            |
| DBSCAN             | eps = 0.5, min_samples = 5 | Identified <b>4 clusters</b> with <b>0 noise (outlier) points</b> .                                  |
| KNN Classifier     | k = 5                      | Achieved the <b>best balance between training and testing accuracy</b> , giving good generalisation. |

**Recommend: which clustering method would you use for this mall's marketing team, and which value of k for the classifier gives the best generalisation? Justify with your numbers.**

Based on the results, I would recommend using K-Means for the mall's marketing team because it divided the customers into 5 clear groups, making it easier to understand different customer behaviours and plan targeted marketing campaigns. In comparison, DBSCAN formed only 4 clusters and did not identify any outliers in this dataset.

For the KNN classifier, I would choose k=5 because it gave almost 100% training and testing accuracy, showing that the model performs well on both training and unseen data. When k was increased to 80, the accuracy dropped to around 29% on the training set and 25% on the testing set, which indicates underfitting. Therefore, k=5 provides the best generalisation for this dataset.

#Part 8 – Critical Thinking Questions

**1. Why does K-Means require you to choose the number of clusters in advance, while DBSCAN does not?**

K-Means requires the number of clusters ((k)) to be specified before training because it groups the data into exactly that many clusters. In contrast, DBSCAN does not require the number of clusters in advance because it forms clusters based on the density of the data points. The number of clusters is determined automatically depending on the data distribution.

###2. DBSCAN labelled some points as noise (-1). Is that a bug or a useful feature? Explain.

It is not a bug; it is a useful feature of DBSCAN. Points labeled -1 are considered noise or outliers because they do not belong to any dense cluster. This helps identify unusual or isolated data points, making DBSCAN effective for datasets that contain outliers.

### **3. A classmate gets 100% train accuracy but only 78% test accuracy. What is happening and how would you fix it?**

The model is overfitting because it has learned the training data too closely and does not generalize well to new data. This can be fixed by choosing a larger value of  $k$ , using more training data, or applying techniques such as cross-validation to improve generalization.

###4. If a model is underfitting, will collecting more data alone usually fix it? Why or why not?

No, collecting more data alone usually does not fix underfitting. Underfitting occurs because the model is too simple to learn the underlying patterns in the data. It can be improved by using a more complex model, selecting better features, or adjusting the model parameters.

###5. Why is feature scaling important for both K-Means and KNN, but not for a Decision Tree?

Feature scaling is important for K-Means and KNN because both algorithms use distance calculations, and features with larger values can dominate the results. Decision Trees do not use distances; they split data based on feature values, so feature scaling is generally not required.

#BONUS

```
In [37]: #Apply 5-fold cross-validation to choose the best k for the KNN
classifier.
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier

k_values = [1, 3, 5, 7, 9, 11]
scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    score = cross_val_score(knn, X, y, cv=5).mean()
    scores.append(score)
    print(f"k={k}, Accuracy={score:.2f}")

best_k = k_values[scores.index(max(scores))]
print("Best k:", best_k)
```

```
k=1, Accuracy=0.99
k=3, Accuracy=0.99
k=5, Accuracy=0.99
k=7, Accuracy=0.99
k=9, Accuracy=0.99
k=11, Accuracy=0.99
Best k: 3
```

---

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).